Evaluating Large Language Models for Web Accessibility Remediation in Real-world Websites

Wajdi Aljedaani[1], Kevin Gautier[2], P D Parthasarathy[3], Colton Clemmer[2]

Saudi Data and Artificial Intelligence Authority[1], University of North Texas[2], Birla Institute of Technology and Science[3]

Introduction

Remediating violations of the **Web Content Accessibility Guidelines** (WCAG) remains a **manual** burden for developers. **Large language models** (LLMs) offer a distinct advantage over automatic testing tools: rather than merely detecting flaws, they can **generate** direct, code-level fixes. Eclaudxisting research focuses on general software engineering tasks, such as security-compliant code generation, functional bug repair, and conversational repair on isolated benchmarks, and these findings may not translate to complex web environments, where **accessibility** depends heavily on semantic layout and user experience. To address this gap, we present an empirical evaluation of **LLM remediation** across multiple WCAG 2.2 success criteria on unmodified, real-world production websites. We seek to understand to what extent can these models **remediate WCAG violations** in production codebases, to what extent different **prompting techniques** affect prompting affect remediation accuracy, and if **combining GPT and LLaMA** outputs improve remediation rates.

Methodology

- **Dataset**: Ten real-world homepages, screened from 30 candidates with WebAIM WAVE and kept only if they had verifiable violations. HTML was downloaded via Playwright and cleaned with BeautifulSoup (378 to 20,552 lines).
- **Models**: GPT-5.5 and LLaMA-3.3 70B Instruct, plus a combined GPT + LLaMA configuration.
- **Prompting strategies**: Zero-Shot, Few-Shot, In-Context Learning, and Chain-of-Thought.
- **Scoring**: Manually inspected patches rated Passing, Partially Passing, or Failing, with a remediation rate giving half-credit for partial fixes. The combined setting counts a violation as fixed if either model passed under any strategy.

Performance of GPT and LLaMA in Remediating WCAG Violations in Real-World Projects

No.	Success Criteria	GPT				LLaMA				GPT Fixing %	LLaMA Fixing %
		ZS	FS	ICL	CoT	ZS	FS	ICL	CoT		
1	Img Element Missing Alt Attribute	11.54%	23.08%	23.08%	25.00%	15.38%	30.77%	23.08%	26.92%	20.67%	27.69%
2	button has no text in label	0%	3.85%	7.69%	5.77%	15.38%	11.54%	11.54%	15.38%	5.00%	13.85%
3	header element is empty	50%	100%	50%	100%	0%	100%	50%	100%	70%	70%
4	input element, type of "text", has no text in label	0%	0%	0%	0%	0%	0%	0%	100%	0%	20%
5	input element, type of "text", missing an associated label	12.50%	87.50%	50%	50%	25%	87.5%	100%	62.5%	52.5%	72.5%
6	anchor contains no text	0%	50%	25%	25%	25%	25%	25%	75%	35%	45%
7	document language not identified	0%	100%	100%	100%	0%	50%	100%	100%	80%	70%
	Total Average	10.58%	52.06%	36.54%	43.68%	11.54%	43.54%	44.23%	68.54%	37.6%	45.58%

- LLaMA performed well on structural, rule-based fixes.
- GPT more sensitive to surrounding code clutter, improved with few-shot prompting.
- LLaMA achieved a higher overall fixing rate (45.58%) than GPT (37.6%)

- Combining the models helped where their errors differed.
- Combining GPT and LLaMA raised the remediation rate to 49.26%, outperforming either model alone.
- When both models hit the same semantic barrier, combining cannot overcome it.

Accessibility Remediation of WCAG Violations Identified in Real-World Projects using GPT + LLaMA

No.	Success Criteria	GPT + LLaMA				Overall Fixing %
		ZS	FS	ICL	CoT	
1	Img Element Missing Alt Attribute	13.46%	26.92%	23.0%	25.96%	24.81%
2	button has no text in label	50%	56.25%	68.75%	62.5%	62.50%
3	header element is empty	25%	100%	50%	100%	70%
4	input element, type of "text", has no text in label	0%	0%	0%	50%	10%
5	input element, type of "text", missing an associated label	18.75%	87.5%	75%	56.25%	62.5%
6	anchor contains no text	12.5%	37.5%	25%	50%	40%
7	document language not identified	0%	75%	100%	100%	75%
	Total Average	17.10%	54.74%	48.83%	63.53%	49.26%

*A violation counts as fixed if either model passed under any of the four strategies

Discussion

Both models failed on semantic tasks such as image descriptions and link text, fabricating elements or generic properties when text clues were missing. This suggests text-only prompts cannot link code to visual layout. Failures followed four patterns: targeting the wrong element, inventing fixes elsewhere in the file, generating excessive markup, and applying valid but unconventional fixes. Because outputs often look correct while introducing new problems, the models are not ready for unsupervised use. This grounding limitation is partly inherited from training data that underrepresents accessibility, so closing the gap will take stronger accessibility education and earlier WCAG emphasis, not better prompting alone.

Implications

Practitioners should use LLM-based remediation as an aid rather than a standalone process, generating candidate patches after automated accessibility testing while the development team keeps the final decision. Tool developers should integrate remediation directly with accessibility checkers so the violation, affected code, and relevant WCAG criterion reach the model automatically, and should encourage focused patches that change only what is needed. Teams with limited accessibility expertise will need stronger validation and review, while experienced developers are better placed to refine model suggestions.